

Student 23 **MALLI CHANDANA** [192472250RD](#)

3 / 3 submitted

Q1. Multiple Threads Using a Single Class

Score: 10.00 TC: 3/3 passed

CODE

```
import java.util.Scanner;

public class Main{
    public static void main(String[] args) throws InterruptedException {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        NumberThread t1 = new NumberThread("Prime", n);
        NumberThread t2 = new NumberThread("Armstrong", n);
        NumberThread t3 = new NumberThread("Reverse", n);

        t1.start();
        t1.join();

        t2.start();
        t2.join();

        t3.start();
        t3.join();
    }
}

class NumberThread extends Thread{
    int n;

    NumberThread(String name, int n){
        super(name);
        this.n = n;
    }

    public void run() {
        if(getName().equals("Prime")){
            boolean prime = true;

            if(n < 2)
                prime = false;

            for(int i = 2; i < n; i++){
                if(n % i == 0){
                    prime = false;
                    break;
                }
            }

            if(prime)
                System.out.println("Prime Thread: " + n + " is a prime number");
            else
                System.out.println("Prime Thread: " + n + " is not a prime number");
        }

        else if(getName().equals("Armstrong")){
            int temp = n;
            int sum = 0;

            while(temp > 0){
                int digit = temp % 10;
                sum = sum + digit * digit * digit;
                temp = temp / 10;
            }
        }
    }
}
```

```
        if(sum == n)
            System.out.println("Armstrong Thread: " + n + " is an Armstrong number");
        else
            System.out.println("Armstrong Thread: " + n + " is not an Armstrong number");
    }

    else if(getName().equals("Reverse")){
        int temp = n;
        int reverse = 0;

        while(temp > 0){
            int digit = temp % 10;
            reverse = reverse * 10 + digit;
            temp = temp / 10;
        }

        System.out.println("Reverse Thread: Reverse of " + n + " = " + reverse);
    }
}
```

INPUT

153

OUTPUT

Prime Thread: 153 is not a prime number
Armstrong Thread: 153 is an Armstrong number
Reverse Thread: Reverse of 153 = 351

Q2. Multiple thread classes

Score: 20.00 TC: 3/3 passed

CODE

```
import java.util.Scanner;

public class Main{
    public static void main(String[] args) throws Exception{
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        PrimeThread t1 = new PrimeThread(n);
        FactorialThread t2 = new FactorialThread(n);
        ReverseThread t3 = new ReverseThread(n);

        t1.start();
        t1.join();

        t2.start();
        t2.join();

        t3.start();
        t3.join();
    }
}

class PrimeThread extends Thread{
    int n;

    PrimeThread(int n){
        this.n = n;
    }

    public void run(){
        boolean prime = true;

        if(n < 2)
            prime = false;

        for(int i = 2; i < n; i++){
            if(n % i == 0){
                prime = false;
                break;
            }
        }

        if(prime)
            System.out.println("Prime: " + n + " is a prime number");
        else
            System.out.println("Prime: " + n + " is not a prime number");
    }
}

class FactorialThread extends Thread{
    int n;

    FactorialThread(int n){
        this.n = n;
    }

    public void run(){
        int fact = 1;

        for(int i = 1; i <= n; i++)
```

```
        fact = fact * i;

        System.out.println("Factorial: " + n + "! = " + fact);
    }
}

class ReverseThread extends Thread{
    int n;

    ReverseThread(int n){
        this.n = n;
    }

    public void run(){
        int temp = n;
        int rev = 0;

        while(temp > 0){
            rev = rev * 10 + temp % 10;
            temp = temp / 10;
        }

        System.out.println("Reverse: Reverse of " + n + " = " + rev);
    }
}
```

INPUT

7

OUTPUT

Prime: 7 is a prime number
Factorial: 7! = 5040
Reverse: Reverse of 7 = 7

Q3. Thread Synchronization

Score: 10.00 TC: 3/3 passed

CODE

```
import java.util.*;

public class Main{
    public static void main(String[] args) throws InterruptedException{
        Counter c = new Counter();

        ThreadA t1 = new ThreadA(c);
        ThreadB t2 = new ThreadB(c);

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("Final Count = " + c.count);
    }
}

class Counter{
    int count = 0;

    synchronized void increment(){
        count++;
    }
}

class ThreadA extends Thread{
    Counter c;

    ThreadA(Counter c){
        this.c = c;
    }

    public void run(){
        for(int i = 0; i < 1000; i++){
            c.increment();
        }

        System.out.println("ThreadA completed");
    }
}

class ThreadB extends Thread{
    Counter c;

    ThreadB(Counter c){
        this.c = c;
    }

    public void run(){
        for(int i = 0; i < 1000; i++){
            c.increment();
        }

        System.out.println("ThreadB completed");
    }
}
```

OUTPUT

```
ThreadA completed  
ThreadB completed  
Final Count = 2000
```